Database Systems(CSL3050)

Lab Assignment 8 - Report

Soumen Kumar
Roll No-B22ES006

1 Introduction

In this assignment, I connected to a MySQL database using Java Database Connectivity (JDBC) and performed several SQL operations. I implemented both static and dynamic queries for inserting, updating, deleting, and fetching records from the database. The following report describes the queries executed, the challenges encountered, and how they were addressed.

2 Development Environment

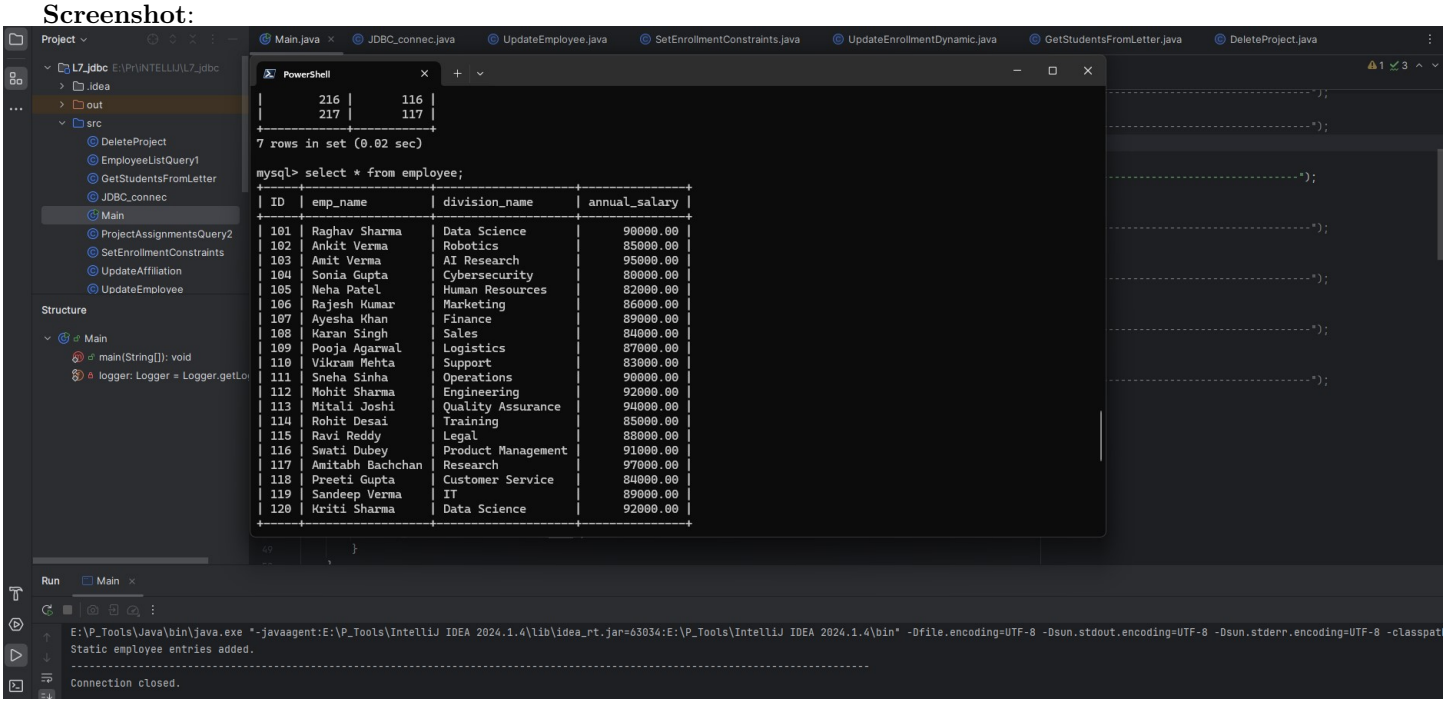
For this assignment, the following technologies were used:

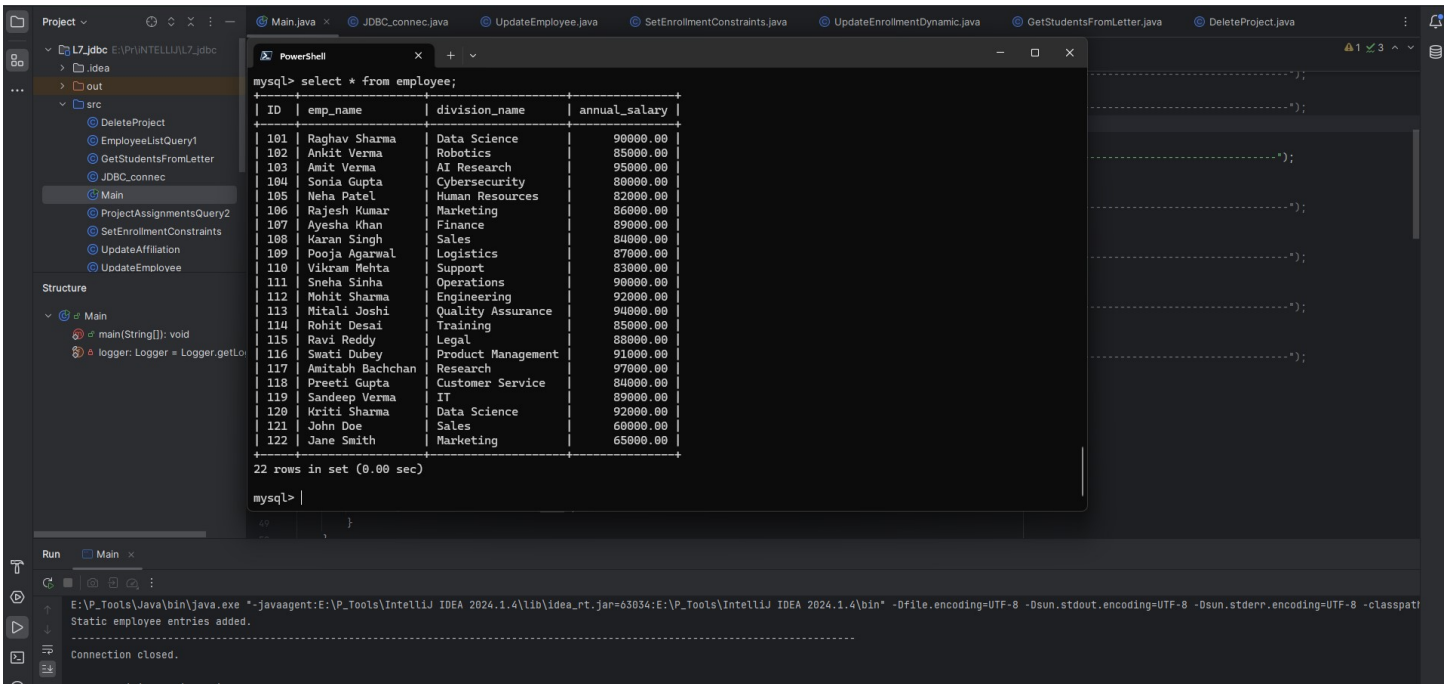
- **IDE**: IntelliJ IDEA
- **Database**: MySQL
- **Programming Language**: Java using JDBC

3 Query 1: Static Update of Employee Table

This query involved using a 'PreparedStatement' to insert static data into the 'Employee' table. Two employee records were inserted statically.

Explanation: This query was executed by manually setting values for each record in the 'PreparedStatement'.
Screenshot:



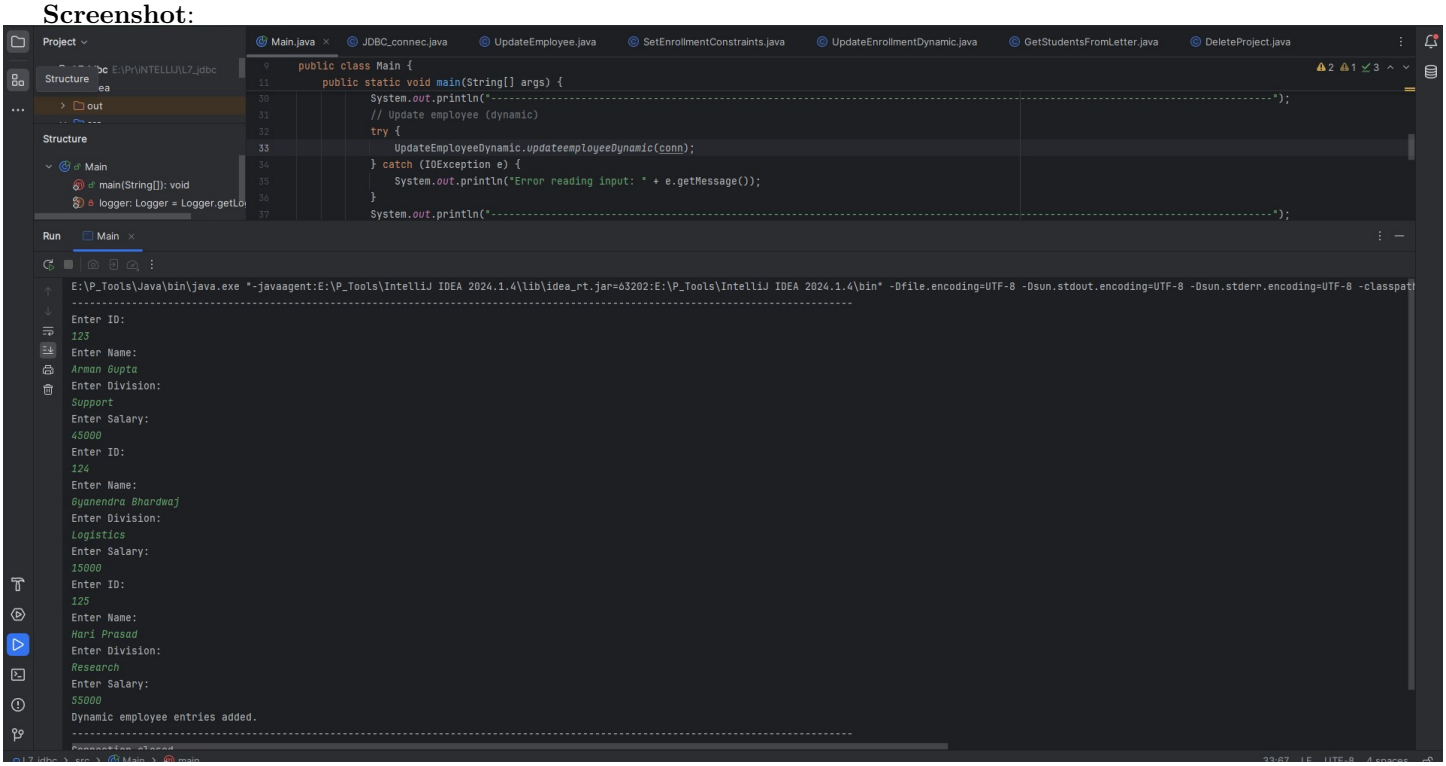


4 Query 2: Dynamic Update of Employee Table

I performed dynamic insertion of employee records by taking user input using 'BufferedReader'. Three employee records were inserted into the 'Employee' table dynamically.

Explanation: This query allowed for dynamic data input, enabling real-time user interaction to insert employee details into the table.

Screenshot:



```
mysql> select * from employee;
+-----+--------------------+--------------------+---------------+
| ID  | emp_name           | division_name      | annual_salary |
+-----+--------------------+--------------------+---------------+
| 101 | Raghav Sharma      | Data Science       |      90000.00 |
| 102 | Ankit Verma        | Robotics           |      85000.00 |
| 103 | Amit Verma         | AI Research        |      95000.00 |
| 104 | Sonia Gupta        | Cybersecurity      |      80000.00 |
| 105 | Neha Patel         | Human Resources    |      82000.00 |
| 106 | Rajesh Kumar       | Marketing          |      86000.00 |
| 107 | Ayesha Khan        | Finance            |      89000.00 |
| 108 | Karan Singh        | Sales              |      84000.00 |
| 109 | Pooja Agarwal      | Logistics          |      87000.00 |
| 110 | Vikram Mehta       | Support            |      83000.00 |
| 111 | Sneha Sinha        | Operations         |      90000.00 |
| 112 | Mohit Sharma       | Engineering        |      92000.00 |
| 113 | Mitali Joshi       | Quality Assurance  |      94000.00 |
| 114 | Rohit Desai        | Training           |      85000.00 |
| 115 | Ravi Reddy         | Legal              |      88000.00 |
| 116 | Swati Dubey        | Product Management |      91000.00 |
| 117 | Amitabh Bachchan   | Research           |      97000.00 |
| 118 | Preeti Gupta       | Customer Service   |      84000.00 |
| 119 | Sandeep Verma      | IT                 |      89000.00 |
| 120 | Kriti Sharma       | Data Science       |      92000.00 |
| 121 | John Doe           | Sales              |      60000.00 |
| 122 | Jane Smith         | Marketing          |      65000.00 |
| 123 | Arman Gupta        | Support            |      45000.00 |
| 124 | Gyanendra Bhardwaj | Logistics          |      15000.00 |
| 125 | Hari Prasad        | Research           |      55000.00 |
+-----+--------------------+--------------------+---------------+
```

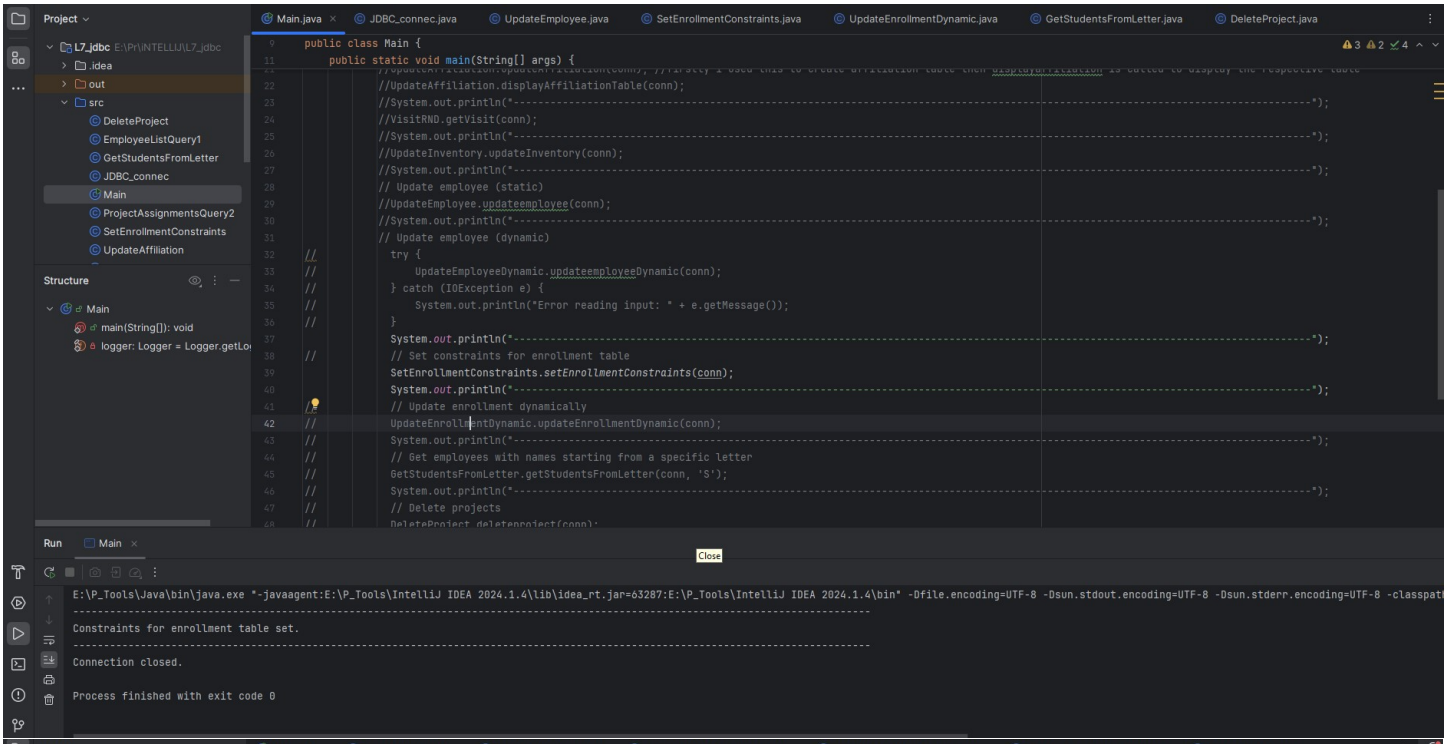
5 Query 3: Set Constraints for Enrollment Table and Dynamic Update

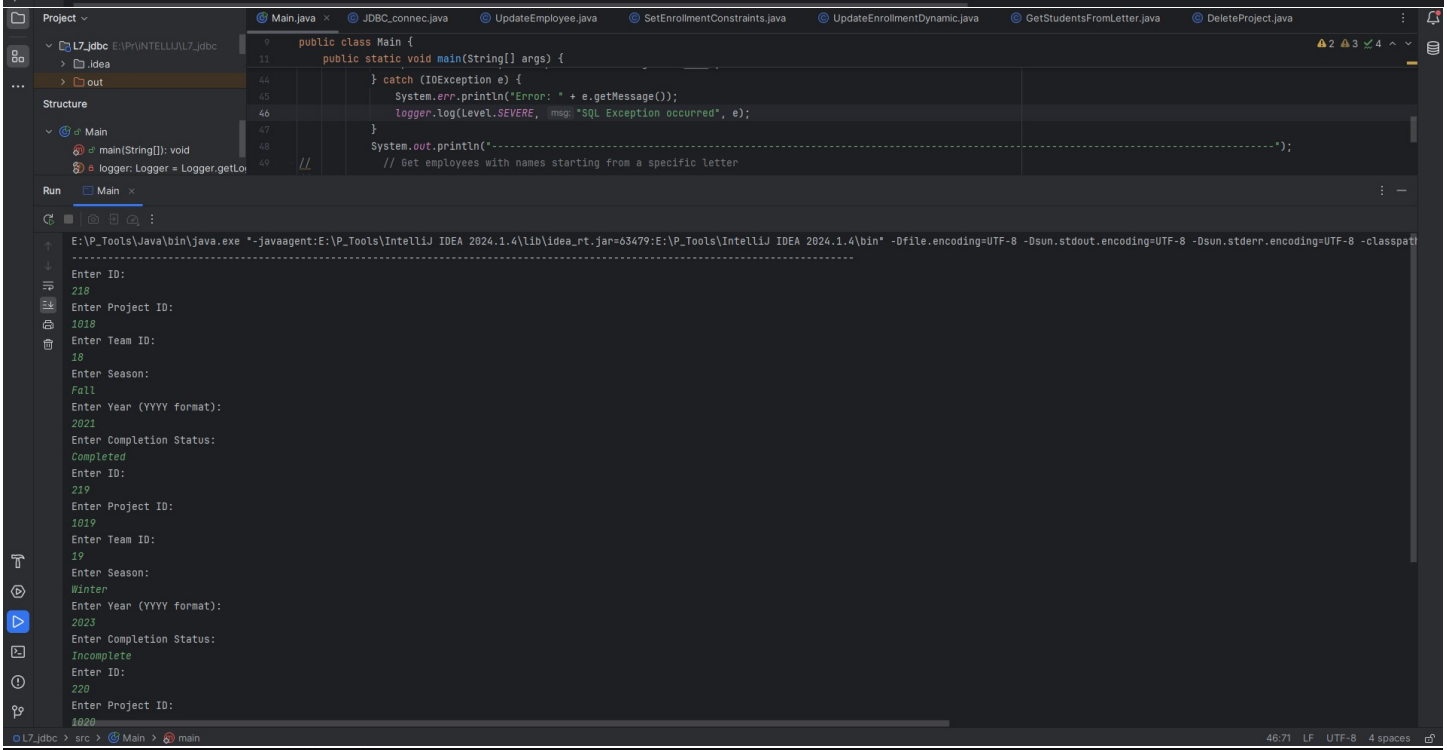
This query modified the constraints on the ‘Enrollment‘ table to ensure data consistency. The ‘StudentID‘ column was constrained to accept only integers, and the ‘Enrollmentdate‘ column was formatted to accept valid date values. After setting the constraints, I dynamically inserted records into the ‘Enrollment‘ table.

Explanation: The constraints were applied to enforce data type rules, and dynamic entries were added to the table without violating these constraints.

Screenshot:

```
mysql> describe enrollment;
+-------------------+-------------+------+-----+---------+-------+
| Field             | Type        | Null | Key | Default | Extra |
+-------------------+-------------+------+-----+---------+-------+
| ID                | int         | NO   | PRI | NULL    |       |
| proj_id           | int         | NO   | PRI | NULL    |       |
| team_id           | int         | NO   | PRI | NULL    |       |
| season            | varchar(20) | NO   | PRI | NULL    |       |
| year              | year        | NO   | PRI | NULL    |       |
| completion_status | varchar(50) | YES  |     | NULL    |       |
+-------------------+-------------+------+-----+---------+-------+
6 rows in set (0.01 sec)
```





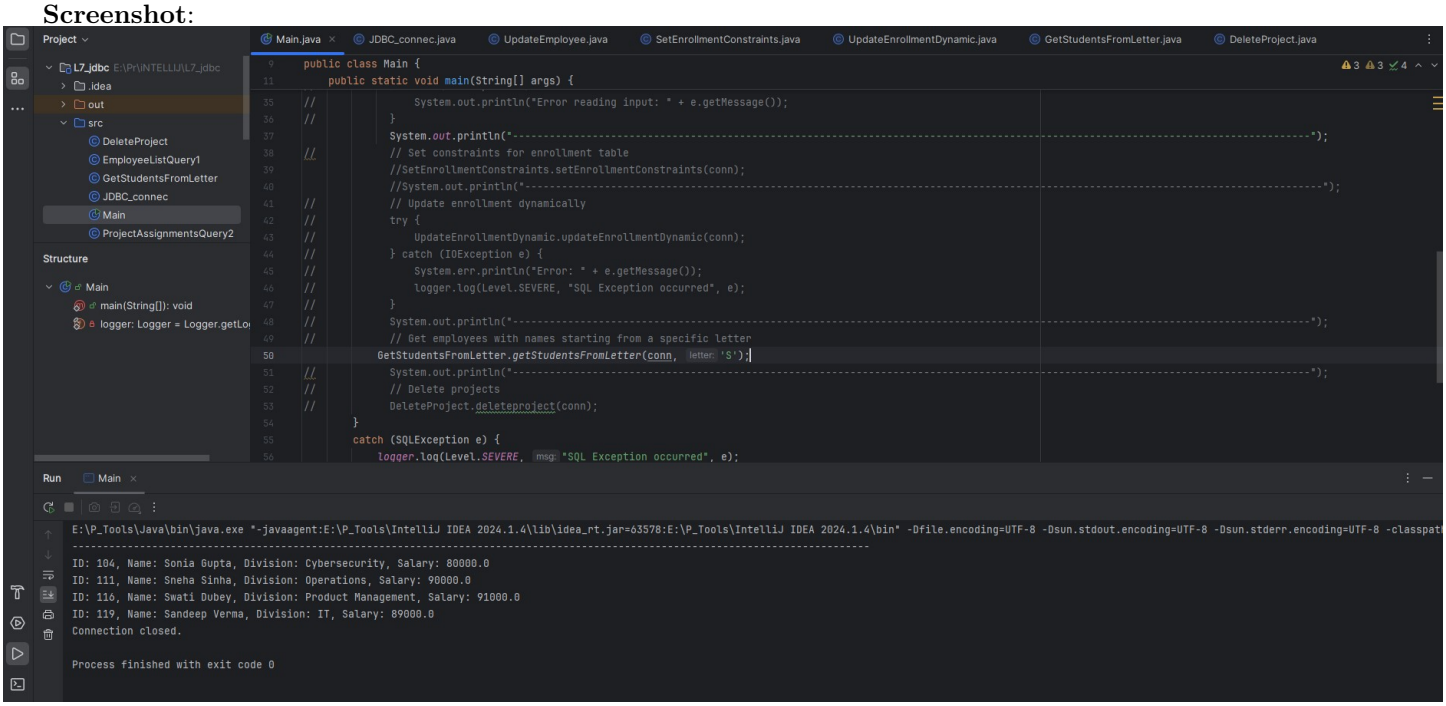

```
mysql> select * from enrollment;
+-----+---------+---------+--------+------+-------------------+
| ID  | proj_id | team_id | season | year | completion_status |
+-----+---------+---------+--------+------+-------------------+
| 211 |    1011 |      11 | Winter | 2021 | Completed         |
| 212 |    1012 |      12 | Spring | 2022 | Incomplete        |
| 213 |    1013 |      13 | Summer | 2023 | Completed         |
| 214 |    1014 |      14 | Fall   | 2022 | Incomplete        |
| 215 |    1015 |      15 | Winter | 2023 | Completed         |
| 216 |    1016 |      16 | Spring | 2022 | Incomplete        |
| 217 |    1017 |      17 | Summer | 2021 | Completed         |
| 218 |    1018 |      18 | Fall   | 2021 | Completed         |
| 219 |    1019 |      19 | Winter | 2023 | Incomplete        |
| 220 |    1020 |      20 | Winter | 2022 | Complete          |
+-----+---------+---------+--------+------+-------------------+
10 rows in set (0.00 sec)
```

6 Query 4: Fetch Students Based on Last Name Initial

This query was designed to retrieve records from the ‘Student‘ table whose last names started with a specific letter. For example, if the input was ’S’, the method would retrieve all students whose last names started with ’S’.

Explanation: The query utilized the SQL ‘LIKE‘ clause to filter records based on the initial letter of the last name.

Screenshot:

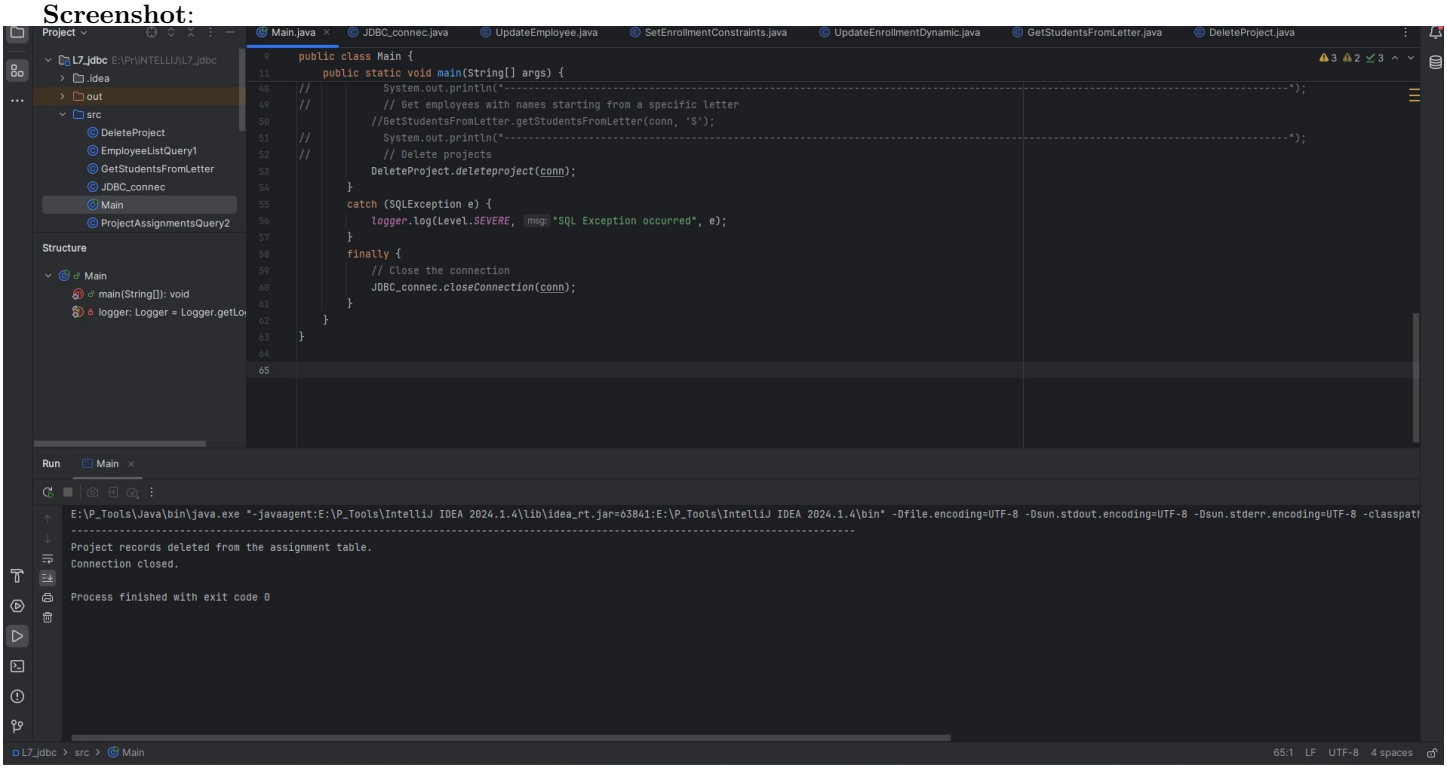


7 Query 5: Delete Project Records

Initially, I attempted to delete records directly from the ‘Project‘ table. However, due to foreign key constraints, this operation failed. Therefore, I modified the query to delete records from the ‘Assignment‘ table instead, ensuring that no foreign key violations occurred.

Explanation: The deletion was performed on the ‘Assignment‘ table to maintain database integrity by avoiding foreign key constraint violations related to the ‘Project‘ table.

Screenshot:



```
mysql> select * from assignment;
+---------+---------+--------+------+---------------+--------------+---------+
| proj_id | team_id | season | year | building_name | floor_number | slot_id |
+---------+---------+--------+------+---------------+--------------+---------+
|    1001 |       1 | Spring | 2022 | Tech Tower    |            1 |     101 |
|    1002 |       2 | Fall   | 2023 | Tech Tower    |            2 |     102 |
|    1004 |       4 | Winter | 2023 | Innovation Hub |            2 |     104 |
|    1005 |       5 | Spring | 2024 | R&D Center    |            1 |     105 |
|    1006 |       6 | Fall   | 2024 | R&D Center    |            2 |     106 |
|    1007 |       7 | Summer | 2024 | Central Office |            1 |     107 |
|    1008 |       8 | Winter | 2024 | Central Office |            2 |     108 |
|    1010 |      10 | Fall   | 2025 | Innovation Hub |            3 |     110 |
|    1011 |      11 | Summer | 2025 | R&D Center    |            3 |     111 |
|    1012 |      12 | Winter | 2025 | Central Office |            3 |     112 |
|    1014 |      14 | Fall   | 2026 | Innovation Hub |            4 |     114 |
|    1015 |      15 | Summer | 2026 | R&D Center    |            4 |     115 |
|    1016 |      16 | Winter | 2026 | Central Office |            4 |     116 |
|    1017 |      17 | Spring | 2027 | Tech Tower    |            5 |     117 |
|    1018 |      18 | Fall   | 2027 | Innovation Hub |            5 |     118 |
|    1019 |      19 | Summer | 2027 | R&D Center    |            5 |     119 |
|    1020 |      20 | Winter | 2027 | Central Office |            5 |     120 |
+---------+---------+--------+------+---------------+--------------+---------+
17 rows in set (0.00 sec)
```

8 Considerations and Changes Made

Throughout the execution of this assignment, several considerations and changes were made to the queries to accommodate the database structure:

- **Change 1: Static and Dynamic Employee Insertion** - The employee records were inserted statically and dynamically using the ‘Employee‘ table, based on the schema from the previous assignment.
- **Change 2: Constraints on Enrollment Table** - I modified the constraints for the ‘Enrollment‘ table to enforce proper data types for the ‘StudentID‘ and ‘Enrollmentdate‘ columns.
- **Change 3: Deletion of Project Records** - Instead of deleting records directly from the ‘Project‘ table (which led to foreign key violations), the deletion was performed from the ‘Assignment‘ table.
- **Change 4: Fetching Students by Last Name Initial** - The query was adapted to retrieve students based on the initial of their last names, with a focus on maintaining consistency with the schema.

Note: These adjustments were necessary due to the constraints and relationships defined in the database schema. By deleting from related tables like ‘Assignment‘ rather than directly from ‘Project‘, I ensured that no foreign key violations occurred, preserving database integrity.

9 Conclusion

This assignment demonstrated how to use JDBC to interact with a MySQL database, including static and dynamic updates, applying constraints, retrieving data, and safely deleting records. The changes made during query execution helped ensure compliance with the database schema and maintain data consistency.